DishonorPredict

COMPLETE ARCHITECTURE, INTEGRATION & TECHNICAL OPERATIONS MANUAL

Target Repository Link:	`https://github.com/romesh09/DishonorPredict.git`
Analytical Components:	`train/train.py & prediction service/predict.py`
Orchestration Layout:	`run_predictions.yaml | train_model.yaml`
Global Infrastructure Key:	`GCP_SERVICE_ACCOUNT_JSON`

---

1. System Architecture & Dual-Sheet Isolation

The `DishonorPredict` platform is designed as an autonomous, serverless machine learning operations ecosystem. It enforces strict separation of concerns by separating the analytical matrix training cycle from live production transaction scoring routines. This isolation prevents data pollution and is achieved by splitting workloads across two entirely separate Google Spreadsheet entities:

- **Historical Raw Ingestion Workbook:** Connects to Spreadsheet ID `1NIVid942fOylEXsqztqVR1akz4nnEZTmv9ryRfsUKj8` (Tab Context: `RAW Data`). This file stores historical ledgers, settlement statuses, and historical clearing data used by the monthly training workflow to construct feature categories and calibrate risk matrices.
- **Operational Production Workbook:** Connects to Spreadsheet ID `18nbA_0B0AHpuAYus-zZbKj_HSYAYpbVTdC83lA_D9ZQ` (Tab Context: `dishonorPredict`). This active tracking sheet contains ongoing, live daily transactions that require algorithmic scanning, risk indexing, and real-time inference feedback.

2. Repository Directory Blueprint

The production system assumes an absolute file tree topology. All custom scripts, action deployment maps, and compiled serialization parameters must reside inside the precise directories outlined below:

```
DishonorPredict/
├── .github/
│   └── workflows/
│       ├── run_predictions.yaml   # Daily automated scoring & manual trigger script
│       └── train_model.yaml       # Monthly automated retraining & manual trigger script
├── prediction service/
│   ├── predict.py                # Main real-time production inference synchronization
engine
│   └── artifacts/                # Persistent serialized model storage layer
│       ├── model_metadata.json   # Computed categorical risk maps and global weights
│       └── rf_dishonor_model.onnx # Transformed Scikit-Learn Random Forest binary file
├── train/
│   └── train.py                  # Analytical feature cleaning and model compilation
pipeline
└── requirements.txt              # Production dependency tracking manifest
```

3. Complete Environment Setup & Integration Guide

Follow these steps sequentially to provision your cloud infrastructure on Google Cloud Platform, establish secure credentials, share spreadsheet permissions, and link them to your GitHub repository context.

1. **Step 1: Project Provisioning within the Google Cloud Console**

   Log into the Google Cloud Console. Click on the project selection dropdown menu at the top left of the main utility bar (located next to the Google Cloud logo). In the upper right corner of the modal window, click **New Project**. Name your project clearly (e.g., `DishonorPredict-Automation`), leave the Organization fields as default, and click **Create**. Wait for the project initialization sequence to complete, then confirm from the top dropdown menu that your newly created project is selected as the active directory workspace.

2. **Step 2: Activating the Google Sheets and Google Drive API Frameworks**

   Open the primary navigation sidebar launcher by clicking the three horizontal lines (☰) at the far top-left corner of your dashboard. Hover over **APIs & Services** and select **Library**. In the central search bar, input `Google Sheets API` and press Enter. Click the matching API card from the results, and click the blue **Enable** button. Once the console finishes loading, return to the API Library screen, search for `Google Drive API`, click its card, and select **Enable**.

3. **Step 3: Creating the Automated Service Account Profile**

   Re-open the primary navigation menu (☰), choose **IAM & Admin**, and click **Service Accounts** from the sub-menu. At the top of the interface dashboard toolbar, click ➕ **Create Service Account**. Complete the initial identification parameters: set the **Service account name** to `sheets-operator-agent` (the system will auto-generate a corresponding unique Service Account ID string). Click **Create and Continue**. Bypass the secondary section ("Grant this service account access to

project") and the tertiary section ("Grant users access to this service account") by selecting **Continue** and then **Done**.

4. **Step 4: Generating and Extracting the Cryptographic JSON Private Key**

   You will be redirected back to the main Service Accounts data index table. Locate the row matching your freshly minted `sheets-operator-agent` profile. Under the **Actions** column on the far right edge of that row, click the three structural options dots (`⋮`) and select **Manage keys**. Click the **Add Key** dropdown button row, choose **Create new key**, select the **JSON** file format choice, and click **Create**. A private credential file with a `.json` extension will download automatically to your workstation. Open this file using a local text editor (such as VS Code or Notepad) to view the raw configuration keys.

   > ⚠️ **CRITICAL SECURITY WARNING**
   >
   > The downloaded JSON key file contains the master private credentials to access your cloud resources. Never upload this file directly to public GitHub branches, commit logs, or shared network spaces. Treat it like an admin password.

5. **Step 5: Granting Spreadsheet Editor Delegations to the Agent Email**

   Look inside the downloaded JSON credential code block, isolate the line mapping key designated `"client_email"`, and copy its corresponding value string (it will read similar to: `sheets-operator-agent@your-project-id.iam.gserviceaccount.com`). Open both of your independent monitoring spreadsheets in a browser window:

   - Historical Ingestion Spreadsheet ID: `1NIVid942fOylEXsqztqVR1akz4nnEZTmv9ryRfsUKj8`
   - Operational Production Spreadsheet ID: `18nbA_0B0AHpuAYus-zZbKj_HSYAYpbVTdC83lA_D9ZQ`

   On each spreadsheet screen, click the bright green **Share** button located in the upper right. Paste your copied service account email address directly into the collaborator user field. Set the permission access role to **Editor**, uncheck the "Notify people" checkbox to prevent communication script execution faults, and click **Share**.

6. **Step 6: Registering Vault Secrets inside GitHub Repository Settings**

   Navigate to your public project repository page on GitHub (`https://github.com/romesh09/DishonorPredict`). Locate the horizontal navigation layout bar immediately below your repository title and click the far-right icon named **Settings** ⚙️. Scroll down along the left sidebar menu index to find the **Security** sub-section block grouping. Click **Secrets and variables** and choose **Actions** from the resulting sublist. Click the green button titled **New repository secret** and define the following input mapping:

   - **Name:** `GCP_SERVICE_ACCOUNT_JSON`
   - **Value:** Select, copy, and paste the entire, unedited raw structural text payload string contained inside your downloaded service account credential JSON private key file. Click **Add secret**.

4. Technical Pipeline Deep-Dive

A. Training & Core Serialization Matrix (train/train.py)

The `train.py` script compiles historical tracking variables, transforms complex fields, and prepares classification maps.

- **Historical Extraction:** Automatically connects to the historical Google Sheet spreadsheet environment to extract real transaction samples.
- **Feature Engineering Mechanics:**
  - Converts raw transaction numeric items safely while resolving missing numbers to zero fields.
  - Parses standard chronological dates (`Collection Date`) to establish time-indexed cyclic factors.
  - Calculates a localized `agent_risk_score` based on historical settlement results for each agent. If a new agent profile appears, the engine assigns a safe overall statistical average risk.
- **Model Construction & Serialization:** Trains a Scikit-Learn `RandomForestClassifier` with balanced weight tracking parameters to mitigate class imbalance. The resulting model transitions immediately into an interoperable ONNX format block saved into `prediction service/artifacts/rf_dishonor_model.onnx`, alongside categorical distributions stored within `model_metadata.json`.

B. Production Execution Engine (prediction service/predict.py)

The live inference engine executes automated verification workflows across production worksheets using an optimized, runtime execution framework.

- **Operational Sheet Synchronization:** Scans active data rows from the tracking sheet, strips previous model artifacts, and handles processing gaps on open rows.
- **Runtime Inference Execution:** Restructures real-time transactional metrics using the previously compiled metadata dictionary. Features pass through a stateless `onnxruntime` scoring session, calculating probability margins without complex server overhead.
- **Target Population Layout:** Writes evaluation parameters back to the live tracker using a `10%` operational danger threshold constraint. Rows exceeding this threshold receive clear warning flags: `Dishonor Probability` (e.g., `42.3%`) and `ML prediction` (set to `🚨 FLAG: High Risk (Review Transaction)`).

5. GitHub Actions Automation & Dual-Trigger Strategies

The deployment framework incorporates dual-trigger operational models inside both active automated orchestration pipelines. They run on standard chronological interval crons, but include `workflow_dispatch` triggers to allow manual overrides via the GitHub user interface anytime.

A. Daily Production Inference Routine (run_predictions.yaml)

- **Automated Cron Schedule:** Executes automatically every single day at `06:00 UTC` (11:30 AM IST).
- **Manual Dispatch Override:** Can be forced manually via the UI to process transaction rows immediately when new ledger rows are input.
- **Processing Mechanics:** Ingests newly written spreadsheet lines, parses localized timestamps, matches agent historical weights against the serialized metadata file, runs values through the compiled ONNX network path, and updates live tracking columns.

B. Monthly Model Optimization Pipeline (train_model.yaml)

- **Automated Cron Schedule:** Triggers automatically on the 1st day of every active calendar month at midnight UTC.
- **Manual Dispatch Override:** Can be run manually at any choice point (e.g., after cleaning immense historical databases) to update modeling coefficients instantly.
- **Repository Access Actions:** Logs into the historical worksheet data table, builds a 100-estimator Scikit-Learn Random Forest Classifier, converts the resulting parameters to open-neural-network-exchange binary graphs, and automatically pushes the optimized model files back into `prediction service/artifacts/` using writing permissions.

6. How to Manually Trigger a Pipeline via GitHub UI

If you need to execute either workflow process immediately without waiting for the next cron window, follow these manual dispatch instructions:

1. Navigate to your main project repository dashboard on GitHub (`romesh09/DishonorPredict`).
2. Click into the top global navigation header index tab labeled **Actions**.
3. On the left-side column list pane, select the target pipeline you want to run: either **"Automated Dishonor Predictions Daily Run"** or **"Model Training and Artifact Compilation Pipeline"**.
4. Look to the right side of the screen over the run histories log index and locate the gray bar configuration row containing a **Run workflow** dropdown button.
5. Click the dropdown, verify your target deployment branch line (usually `main`), and click the green **Run workflow** button to launch the pipeline instance instantly.

7. Local Sandbox Workspace Deployment Verification

To run sanity checks or execute your python scripts locally on a personal development machine, execute these terminal operations:

```
# 1. Initialize Virtual Isolation Environment and Install Manifest
git clone https://github.com/romesh09/DishonorPredict.git
cd DishonorPredict
python -m venv venv
source venv/bin/activate  # Windows terminal execution target: venv\Scripts\activate
pip install -r requirements.txt

# 2. Map Google Service Account Credentials Environment Variable Locally
# For macOS/Linux terminals:
export GCP_SERVICE_ACCOUNT_JSON='{"type": "service_account", "project_id": "your-
id", ...}'
# For Windows PowerShell terminals:
$env:GCP_SERVICE_ACCOUNT_JSON='{"type": "service_account", "project_id": "your-id", ...}'

# 3. Verify Model Building, Metric Extraction, and ONNX Compilation Mechanics
python train/train.py

# 4. Verify Production Sheet Verification, Data Engineering, and Prediction
Synchronization
python "prediction service/predict.py"
```

8. Unified Project Dependencies Manifest

Below is your production requirement schema. These packages are configured to support both training transformations and production model execution operations:

```
# ============================================================================
# DishonorPredict - Unified Project Dependencies
# Clean dependencies for data ingestion, training, and ONNX deployment layers.
# ============================================================================

# Data Manipulation & Array Operations
pandas>=2.0.0
numpy>=1.24.0

# Google Sheet Ingestion & Ecosystem Security
gspread>=5.10.0
google-auth>=2.20.0
google-oauth2-tool>=0.0.1

# Core Machine Learning Matrix & Serialization Engine (Used in train.py)
scikit-learn>=1.2.0
skl2onnx>=1.14.0

# Production Model Inference Session Engine (Used in predict.py)
onnxruntime>=1.15.0
```